Latent trait layer. For each real user u (not account), sample:

- **Wealth** $W_{u,0}$ from a heavy-tailed distribution (log-normal or Pareto are typical) to represent heterogeneity.
- **Risk aversion** A_u (e.g. CRRA coefficient) to control the stake fraction.
- **Signal quality and miscalibration** τ_u (precision) plus bias parameters for mean bias and over/under-confidence.
- **Participation propensity parameters** (baseline rate plus burstiness).
- **Optional manipulation parameters** κ_u: external incentives, sybil budget, collusion group.

Participation is not "missing data"; it is often strategic. Useful models:

- **Baseline availability:**
$$a_{u,t} \sim \mathrm{Bernoulli}(p_u).$$
- **Bursty sessions:** clustered participation rather than IID.
- **Edge-threshold entry:** act only when disagreement is large enough.
- **Avoid skill decay:** act only when expected score is high (protect reputation/skill estimate).

To generate realistic report distributions you need more than "skill = noise".

Latent traits to include.

- **Signal precision** (expected loss level).
- **Bias and miscalibration** (mean bias, over/under-confidence).
- **Correlation across agents** (common information sources).
- **Concept drift exposure and heterogeneous adaptation rates**.

Example: correlated error structure (common shock).

$$\epsilon_{u,t} = \rho\, \xi_{g,t} + \sqrt{1-\rho^2}\, \nu_{u,t}.$$

$$W_{i,t+1} = \max\big(0,\ W_{i,t} + \pi_{i,t}\big) + \mathrm{topup}_{i,t} - \mathrm{withdraw}_{i,t}.$$

Meaning. A rule that maps state and beliefs to stake size:

$$b_{i,t} = \min\big(W_{i,t},\, b_{\max},\, W_{i,t} f_{i,t}\big),$$

where $f_{i,t} \in [0, 1]$ is the fraction of bankroll risked.

Common policies.

- **Fixed fraction:**
$$f_{i,t} = f_i.$$
Produces stable proportional betting.

- **Edge-proportional:**
$$f_{i,t} = \mathrm{clip}\big(k \cdot \mathrm{edge}_{i,t},\ 0,\ 1\big).$$
"Edge" is an internal estimate of advantage, e.g. expected score gain vs a baseline.

- **Kelly-like:** choose $f_{i,t}$ to maximise expected $\log W_{i,t+1}$. In practice, approximate with an edge-proportional rule plus a safety multiplier.

- **Capped:** hard limits $b_{\max}$ or per-round weight caps interact with strategic behaviour.

Per-round interface. Per round t, the behaviour module returns a standard submission set. For each account i:

- **Account id:** i.

- **Participation flag:** $a_{i,t} \in \{0, 1\}$.

- **Report:** $r_{i,t} \in \mathcal{R}$ (point forecast or distribution object).

- **Deposit / stake:** $b_{i,t} \in R_{\geq 0}$.

Round input. Define the round input as

$$\mathrm{RoundInput}_t = \big\{\big(i,\ a_{i,t},\ r_{i,t},\ b_{i,t},\ z_{i,t}\big)\big\}_{i=1}^{N}.$$

Validity rules. If $a_{i,t} = 0$, then the submission is treated as absent:

$$a_{i,t} = 0 \ \Rightarrow\ b_{i,t} = 0 \text{ and } r_{i,t} = \ \text{(ignored)}.$$

Enforce:
$$0 \leq b_{i,t} \leq W_{i,t}.$$

The core does not interpret $z_{i,t}$ unless explicitly wired in; otherwise it is logging-only.

Core signature. The core consumes only $(\mathrm{state}_t,\ \mathrm{RoundInput}_t,\ y_t)$:

$$\mathrm{CoreMechanism}\big(\mathrm{state}_t,\ \mathrm{RoundInput}_t,\ y_t\big)\ \rightarrow\ \big(\mathrm{state}_{t+1},\ \mathrm{logs}_t\big).$$

Key point. The core does not need to know why $\big(a_{i,t}, r_{i,t}, b_{i,t}\big)$ were chosen. It only applies settlement, aggregation, and state updates.

Experiment 2: `deposit_policies` ("Which deposit policy is best?")

Design. Fix the weight rule (mechanism) and vary only the deposit policy.

Metric. Warm-start CRPS (lower is better), averaged over $S = 5$ random seeds.

Mechanism + deposit policy	CRPS
IID-Exp	0.0459
Fixed-Unit	0.0424
Oracle-Precision	0.0229
Bankroll-Confidence	0.0251

Table 1: Experiment 2 results: warm-start CRPS over $S = 5$ seeds.

What this shows.

- **IID-Exp is worst** (0.0459): random deposits inject noise into wagers and degrade performance.
- **Fixed-Unit helps** (0.0424): removing deposit noise improves CRPS, so the mechanism can rely on learned skill rather than noisy stake.
- **Bankroll-Confidence is a large gain** (0.0251): relative improvement over Fixed-Unit is

  $$\frac{0.0424 - 0.0251}{0.0424} \;=\; 0.408 \;\approx\; 41\%.$$

  This indicates that a confidence proxy derived from quantile widths provides informative deposits without oracle access.
- **Oracle-Precision is the ceiling** (0.0229): it uses the true precision signal. Bankroll-Confidence is within

  $$\frac{0.0251 - 0.0229}{0.0229} \;=\; 0.096 \;\approx\; 9.6\%$$

  of this oracle benchmark, despite not observing τ_i.

Ordering.

$$\text{Oracle-Precision} \;<\; \text{Bankroll-Confidence} \;\ll\; \text{Fixed-Unit} \;<\; \text{IID-Exp}.$$

Regime	How deposits are set	Realism
Random	$b \sim \mathrm{Exp}(1)$, independent of forecast quality	Worst case: deposits are pure noise.
Fixed	$b = 1$ for all forecasters	Eliminates deposit signal entirely.
Correlated	$b \propto 1/\tau^2$ (oracle-informed)	Best case: deposits perfectly match precision.
Bankroll	Deterministic: wealth $\times$ confidence; skill-gated with exponent $\eta = 2$; capped	Realistic: no oracle, no exogenous noise.

The four deposit regimes.

- Stake comes from wealth, so it is stable rather than exogenous noise.
- Stake responds to forecast width, so it carries behavioural meaning.

1. **Wealth** sets what someone can afford.
2. **Confidence** (from their own forecast width) sets what they choose to put in.

Step-by-step each round
Step 0: Each person has a wallet. Each forecaster i has wealth $W_{i,t}$, interpreted as an account balance. After the round, wealth updates with profit:

$$W_{i,t+1} = \max\big(0,\ W_{i,t} + \pi_{i,t}\big).$$

Deposits are therefore budget-feasible and not exogenous.

Step 1: They submit a forecast distribution (quantiles). They submit quantiles $q_{i,t}(0.1), q_{i,t}(0.5), q_{i,t}(0.9)$, etc. You interpret how "sure" they are via forecast width:

- wide distribution $\Rightarrow$ "I am not sure",
- narrow distribution $\Rightarrow$ "I am sure".

A simple width statistic is

$$\Delta_{i,t} = q_{i,t}(0.9) - q_{i,t}(0.1).$$

Convert width into a bounded confidence score $c_{i,t}$:

$$c_{i,t} = \mathrm{clip}\Big(\exp(-\beta_c \Delta_{i,t}),\ c_{\min},\ c_{\max}\Big).$$

Thus smaller $\Delta_{i,t}$ yields larger $c_{i,t}$, but confidence is prevented from becoming arbitrarily large.

Step 2: Deposit b is chosen from wealth and confidence only. Define the deposit:

$$b_{i,t} = \min\Big\{W_{i,t},\ f\, W_{i,t}\, c_{i,t},\ b_{\max}\Big\}.$$

Interpretation:

- $fW_{i,t}$ is "I am willing to risk about a fraction f of my wealth",
- $c_{i,t}$ shifts the deposit up or down depending on self-reported confidence,
- the deposit is capped by current wealth and a hard maximum $b_{\max}$.

This removes "random stake noise" because stake is driven by a budget constraint and the forecaster's own report.

Step 3: Convert deposit into effective wager m using skill. Even if someone is rich, if they are bad they should not have much influence. Define the effective wager:

$$m_{i,t} = b_{i,t}\Big(\lambda + (1-\lambda)\sigma_{i,t}^{\eta}\Big).$$

Interpretation:

- $b_{i,t}$ is what they put on the table,
- $\lambda + (1-\lambda)\sigma_{i,t}^{\eta}$ is the "how much of this counts" factor,
- if $\sigma_{i,t}$ is low then $\sigma_{i,t}^{\eta}$ is small and $m_{i,t}$ is suppressed,
- if $\sigma_{i,t}$ is high then $m_{i,t} \approx b_{i,t}$.

The refund is the part that does not count:

$$\text{refund}_{i,t} = b_{i,t} - m_{i,t} \geq 0.$$

Hence "rich-but-bad" agents cannot dominate because their $m_{i,t}$ is gated by low $\sigma_{i,t}$.

Step 4: Use m for both settlement and aggregation.

1. **Settlement** (who wins money) uses $m_{i,t}$.
2. **Aggregation** (whose forecast gets more weight) uses $m_{i,t}$.

The same effective wager therefore controls both payoffs and influence.

Why this solves the problem
Before (undesirable).

- stake is random exponential noise,
- weights are noisy and can be dominated by chance stake spikes,
- a rich agent can dominate by stochastic stake variation.

After (desired).

- stake comes from wealth so it is stable, not noise,
- stake responds to forecast width so it has meaning,
- skill gates how much stake counts so rich-but-bad cannot dominate.

Practical safety valve (recommended). Add a hard cap on per-round influence:

$$\tilde{m}_{i,t} = \frac{m_{i,t}}{\sum_j m_{j,t}}, \qquad \tilde{m}_{i,t} \le \omega_{\max}.$$

Data-generation options for learning skill

Motivation. The motivation is to address a mismatch between two strands of existing work. **Self-financed, proper-scoring wagering mechanisms** provide principled incentives (truthful reporting under a proper score, and controlled transfers), but they typically treat participant influence as essentially static (often tied to stake) and do not model how reliability changes over time. Conversely, **online forecast aggregation** learns time-varying weights from predictive performance, but it usually assumes forecasters are non-strategic and consistently available, and it does not endogenise payments or incentive constraints. In realistic forecasting settings, contributors are heterogeneous, may submit only intermittently, and may respond strategically to how influence and rewards are allocated. Under these conditions, fixed stake-weighting can confound wealth with information quality and can be brittle under distributional drift.

Contribution. This project differs by explicitly **coupling** a self-financed wagering layer with an **online learning layer** that estimates and updates skill from realised outcomes, and uses it to adjust aggregation weights and reward allocation over time, while treating missing and intermittent submissions as a first-class design constraint. The goal is not to claim universally superior forecasting, but to produce a coherent mechanism where (i) incentives are grounded in proper scoring and feasible transfers, and (ii) influence is allowed to evolve with measured performance rather than being fixed by stake alone.

Option A: current generator (truth plus additive noise on $[0, 1]$).

$$y_t \sim \mathrm{Unif}(0,1), \qquad r_{i,t} = \mathrm{clip}\big(y_t + \varepsilon_{i,t},\, 0,\, 1\big), \qquad \varepsilon_{i,t} \sim \mathcal{N}(0, \nu_i^2).$$

$\mathrm{r}_{i,t} = \mathrm{clip}\big(y_t + \varepsilon_{i,t},\, 0,\, 1\big), \qquad \varepsilon_{i,t} \sim \mathcal{N}(0, \nu_i^2)$. Quantiles are built as

$$q_{i,t}(\tau) = \mathrm{clip}\big(\mu_{i,t} + \sigma_i \Phi^{-1}(\tau),\, 0,\, 1\big), \qquad \tau \in (0,1).$$

Pros: simple, fast.
Problems for "learn true skill":

- clipping creates edge artefacts that distort MAE and CRPS behaviour near 0 and 1;

- the point report $\mu_{i,t}$ is a noisy copy of realised y_t, not a coherent predictive distribution;
- quantiles require monotonicity fixes (post hoc) due to clipping.

Option B: exogenous latent truth + naive "noise around truth" (no Bayes step). Let latent truth be exogenous, e.g.

$$z_t \sim \mathcal{N}(0,1) \quad \text{or} \quad z_t = \varphi z_{t-1} + u_t, \;\; u_t \sim \mathcal{N}(0, \sigma_u^2).$$

Map to the unit interval via a link $g : R \to (0,1)$, e.g. probit $g(z) = \Phi(z)$ or logistic $g(z) = \frac{1}{1+e^{-z}}$:

$$y_t = g(z_t), \qquad r_{i,t} = g(z_t + \eta_{i,t}), \qquad \eta_{i,t} \sim \mathcal{N}(0, \tau_i^2),$$

and build quantiles similarly. **Pros:** avoids clipping; truth is exogenous.
Problem: reports are not Bayes-consistent, so $r_{i,t}$ need not be the MAE-optimal conditional median, and $q_{i,t}(\tau)$ need not equal the true conditional quantiles of Y_t given forecaster information.

Option C (recommended): exogenous latent truth + Bayes-consistent forecasters. Use an explicit signal model:

$$Z_t \text{ exogenous}, \qquad X_{i,t} = Z_t + b_i + \varepsilon_{i,t}, \qquad \varepsilon_{i,t} \sim \mathcal{N}(0, \tau_i^2).$$

Choose a prior for Z_t (IID or AR(1)); the conditional distribution $Z_t \mid X_{i,t}$ is Gaussian under a Gaussian prior. With probit link $Y_t = \Phi(Z_t)$, the forecaster's predictive quantiles have a closed form:

$$q_{i,t}(\tau) = \Phi\big(\mu_{i,t}^Z + \sigma_{i,t}^Z \, \Phi^{-1}(\tau)\big),$$

where $(\mu_{i,t}^Z, (\sigma_{i,t}^Z)^2)$ are the posterior mean and variance of $Z_t \mid X_{i,t}$. The MAE-optimal point forecast is the conditional median, and for a monotone link it is

$$r_{i,t} = \text{median}(Y_t \mid X_{i,t}) = \Phi(\mu_{i,t}^Z).$$

Why this is the clean benchmark:

- point reports are conditionally median-optimal (MAE-consistent);
- quantiles are conditionally quantile-optimal (pinball-consistent), and align with CRPS-hat evaluation;
- no clipping artefacts and quantile monotonicity holds by construction;
- "intrinsic skill" corresponds directly to expected proper score under a correctly specified information model.

EXTRA Must enforce:

- $\sigma_{i,t}$ is fixed before forecasters choose $r_{i,t}$ in round t.
- so $\sigma_{i,t}$ is measurable w.r.t. information up to $t-1$.

Effective wager:

$$m_{i,t} := b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big).$$

So $m_{i,t} \le b_{i,t}$, and the tight lower bound is

$$m_{i,t} \ge b_{i,t}\big(\lambda + (1-\lambda)\sigma_{\min}\big) \ge \lambda b_{i,t}.$$

Refund (immediate):

$$\text{refund}_{i,t} = b_{i,t} - m_{i,t} = b_{i,t}(1-\lambda)(1-\sigma_{i,t}).$$

$$\hat{m}_{i,t} = \frac{m_{i,t}}{\sum_{j\in I_t} m_{j,t}}.$$

Linear opinion pool:

$$\hat{r}_t(x) = \sum_{i\in I_t} \hat{m}_{i,t}\, r_{i,t}(x).$$

Quantile averaging:

$$\hat{q}_k^{r_t} = \sum_{i\in I_t} \hat{m}_{i,t}\, q_k^{r_{i,t}}.$$

Assume a strictly proper scoring rule $S(r,\omega)$.

$$s(r,\omega) = 1 - \text{CRPS}(r,\omega),$$

A) Pure skill multiplier (linear)

$$m_i = b_i\, \sigma_i$$

Pros

- Very clean: just rescales wagers.
- Preserves settlement proofs if σ_i is pre-event and report-independent.

Cons

- Zero-skill free option: $\sigma_i \approx 0 \Rightarrow m_i \approx 0$, so downside can vanish.
- Whales still dominate when $\sigma_i \approx 1$: influence and risk scale linearly in b_i.
- Sybil risk if users can obtain multiple identities with different σ (or reset skill).

General Context

A) Pure skill multiplier (linear)

$$m_i = b_i \, \sigma_i$$

Pros

- Very clean: just rescales wagers.
- Preserves settlement proofs if σ_i is pre-event and report-independent.

Cons

- Zero-skill free option: $\sigma_i \approx 0 \Rightarrow m_i \approx 0$, so downside can vanish.
- Whales still dominate when $\sigma_i \approx 1$: influence and risk scale linearly in b_i.
- Sybil risk if users can obtain multiple identities with different σ (or reset skill).

B) Percentage always at risk (current pick)

$$m_i = b_i\big(\lambda + (1-\lambda)\sigma_i\big), \qquad \lambda \in (0,1).$$

Pros

- Guarantees non-trivial downside: even $\sigma_i = 0$ gives $m_i = \lambda b_i$.
- Keeps a single knob λ controlling how "permissionless" the market feels.
- Still fits Raja settlement directly by replacing m_i.

Cons

- It can advantage whales in the usual way: m_i is still linear in b_i, so a large bankroll can buy large exposure (especially if they can keep σ high).
- If λ is small, low-skill can buy influence cheaply: they risk only λ per unit of "headline stake".
- Still vulnerable to skill farming / reset unless σ is hard to manipulate.

Prediction markets are information markets where participants trade in contracts whose payoff depends on the outcome of unknown future events, such as elections or sports results. These markets aggregate information by pulling together heterogeneous signals, models and beliefs into a single probabilistic view of the future. In a well-functioning prediction market, this "wisdom of the crowds" will be the best predictor of the event and is as good, or better than, conventional polling and expert judgement [2].

Forecast elicitation mechanisms ask sellers to report their beliefs concerning a future event. After the event occurs, forecasters are ranked according to the quality of their forecasts and evaluated by a scoring rule. Rewarding only the best forecaster matches many real-world forecasting settings, but ignores the fact that other participants can still provide valuable information.

C) Fixed amount always at risk (absolute floor)

Two common variants:

C1. Additive floor

$$m_i = c + b_i\sigma_i, \qquad c > 0 \ \text{(and require } b_i \geq b_{\min}\text{)}$$

C2. Max floor

$$m_i = \max\big(c,\, b_i\sigma_i\big)$$

Pros

- Ensures everyone risks something meaningful, even if b_i is tiny.
- Makes "play for free" harder than (A), often harder than (B) if c is material.

Cons

- Harsh on small users (participation friction).
- Can create awkward UX: users must understand they are paying a fixed "bond".
- Still linear in b_i beyond the floor, so whales still dominate at scale.

State of Affairs

Combining multiple forecasts produced from a single target series is widely used to improve accuracy through the integration of information gleaned from different sources. Combination schemes have evolved from simple combination methods without estimation to sophisticated methods involving time-varying weights, nonlinear combinations, correlations among components, and cross-learning. A forecast aggregation problem involves combining multiple, noisy predictions of the same outcome into a single forecast that is more accurate, better calibrated and more robust than its constituents. Combining two unbiased forecasts with imperfectly correlated errors reduces mean squared error relative to either forecast alone. In many applications, simple equal-weighted averages perform at least as well as statistically "optimal" weight estimates, because the latter are fragile in the presence of estimation error and non-stationarity. This is called the *forecast combination puzzle* [6].

D) Concave scaling in stake

$$m_i = h(b_i)\big(\lambda + (1-\lambda)\sigma_i\big)$$

with h increasing and concave, eg $h(b) = b^\alpha$ or $h(b) = \log(1 + b/b_0)$.

Pros

- Directly attacks whale dominance: doubling bankroll less than doubles exposure.
- Keeps settlement algebra if you just substitute m_i.

Cons (big)

- Sybil breakage by stake splitting: with concave h and $h(0) = 0$, typically

$$h(b_1) + h(b_2) \geq h(b_1 + b_2),$$

  so splitting one bankroll across many accounts increases total exposure.
- Needs strong identity binding to be safe.

.

E) Capped effective wager

$$m_i = \min\big(b_i(\lambda + (1-\lambda)\sigma_i),\, m_{\max}\big)$$

Pros

- Hard cap on per-account dominance.
- Simple and intuitive.

Cons

- Sybil by account splitting is trivial unless you cap per person.
- If $m_{\max}$ is binding often, you distort incentives for strong forecasters with capital.

F) Skill affects only aggregation (not payoffs)

Use plain wagering with $m_i = b_i$ in Raja.

Compute the forecast aggregation using skill.

Pros

- Settlement incentives remain exactly classical wagering.
- Skill impacts information quality without entangling money flows.

Cons

- Does not solve "rich but inaccurate can dominate" inside the wagering layer.

- Breaks the system

G) Skill affects only the utility pot U

Keep $m_i = b_i$ in wagering, but allocate U with skill

Pros

- You can reward long-run skill without distorting core wagering incentives.
- Easy to tune: size of U controls how strong skill rewards are.

Cons

- Limited effect unless U is large.
- If g depends on b, whales can still pull utility rewards.

H) Random high-stakes betting (lottery-style scaling)

One canonical form: use a small base wager m, then with probability p settle as if wager were m/p (so expected incentives match higher stake).

Pros

- Lets small players access "high-stake-like" incentives without locking big capital.
- Preserves incentives in expectation.

Cons

- High variance outcomes, worse user experience.
- Many nice guarantees become in expectation, not per-realisation.

Expected Results and Impact

1 Make skill "absolute", not relative

Avoid "everyone normalised to sum to 1". Instead keep a per-user reliability score and map it to σ directly.

Example:

Per-round loss $\ell_{i,t}$ from a proper score (or proper loss).

Exponentially weighted moving average:

$$L_{i,t} = (1-\rho)L_{i,t-1} + \rho\,\ell_{i,t}.$$

Map to $[\sigma_{\min}, 1]$:

$$\sigma_{i,t} = \sigma_{\min} + (1 - \sigma_{\min}) \exp(-\gamma L_{i,t}).$$

This makes σ interpretable on its own and stops "relative rank" games.

2 Exposure-weighted learning

Early performance should not buy high σ.

Practical pattern:

Start everyone at $\sigma_{\min}$.

Update only after meaningful exposure, eg weight the update by a capped function of the effective wager:

$$L_{i,t} = (1 - \rho) L_{i,t-1} + \rho\, w(m_{i,t})\, \ell_{i,t}, \qquad w(m) = \min\left(1, \frac{m}{m_{\text{ref}}}\right).$$

So tiny bets do not move skill much, but huge bets do not allow instant farming either (cap).

3 Make skill increase slowly, but drop quickly

Use asymmetric learning rates:

$$\sigma_{i,t+1} = \begin{cases} \sigma_{i,t} + \eta^{\uparrow}\, \Delta_{i,t} & \text{if performance improves} \\ \sigma_{i,t} - \eta^{\downarrow}\, \Delta_{i,t} & \text{if performance worsens} \end{cases} \qquad \text{with } \eta^{\uparrow} \ll \eta^{\downarrow}.$$

Or hard caps:

$$\sigma_{i,t+1} \le \sigma_{i,t} + \Delta_{\max}.$$

This directly targets "farm quickly, cash out later".

4 Ensure "still risk a certain amount"

Your mechanism's actual maximum downside is governed by m, so pick floor parameters that match your risk philosophy:

Set λ not tiny, and enforce $\sigma \ge \sigma_{\min}$:

$$m_i = b_i(\lambda + (1 - \lambda)\sigma_i), \qquad \sigma_i \in [\sigma_{\min}, 1].$$

Add an absolute minimum wager (cleaner than weird micro-bets):

$$m_i \ge m_{\min}.$$

This aligns with your own stated motivation that low-skill should not "play for free", while keeping the Lambert/Raja settlement algebra intact.

$$\Pi_{i,t}(r_t, m_t, \omega_t) = m_{i,t} \left(1 + s(r_{i,t}, \omega_t) - \frac{\sum_{j \in I_t} m_{j,t}\, s(r_{j,t}, \omega_t)}{\sum_{j \in I_t} m_{j,t}} \right).$$

Utility:

Let the client baseline score be

$$s_{c,t} := s(r_{c,t}, \omega_t).$$

Define

$$\tilde{s}_{i,t} := \mathbf{1}\{s(r_{i,t}, \omega_t) > s_{c,t}\}\, s(r_{i,t}, \omega_t).$$

Overall payoff:

$$\hat{\Pi}_{i,t} = \Pi_{i,t} + \mathbf{1}\{U_t > 0\} \left(\frac{\tilde{s}_{i,t}\, m_{i,t}}{\sum_{j \in I_t} \tilde{s}_{j,t}\, m_{j,t}} U_t \right).$$

Cashflow (what a user receives):

$$\text{cashout}_{i,t} = \text{refund}_{i,t} + \hat{\Pi}_{i,t}.$$

Net profit vs the initial deposit $b_{i,t}$ is:

$$\text{profit}_{i,t} = \text{cashout}_{i,t} - b_{i,t} = \hat{\Pi}_{i,t} - m_{i,t}.$$

Gantt Chart

If $\sum_{j \in I_t} m_{j,t} = 0$, the competitive term's denominator is 0. Convention: run no market round (return $r_{c,t}$, and set $\Pi_{i,t} = 0$ for all i).

If $\sum_{j \in I_t} \tilde{s}_{j,t} m_{j,t} = 0$, set the utility term to 0 for all i.

References

You want $\sigma_{i,t}$ to be an absolute reputation signal, not a relative mixture weight. The simplest safe design is:

Choose a per-round loss $\ell_{i,t}$ derived from the same evaluation idea

Two clean options:

Option A (directly from Raja score): if your score is bounded, define

$$\ell_{i,t} := 1 - s(r_{i,t}, \omega_t) \in [0, 1].$$

Option B (pinball loss on quantiles, Vitali setup): for quantile level τ,

$$L^{(\tau)}(y, \hat{y}) = \tau(y - \hat{y})\mathbf{1}\{y \geq \hat{y}\} + (1 - \tau)(\hat{y} - y)\mathbf{1}\{y < \hat{y}\}.$$

Then take $\ell_{i,t}$ as an average over $\tau \in T$, possibly normalised by a scale of y.

Handle intermittent participation explicitly

Let $a_{i,t} \in \{0, 1\}$ indicate whether forecaster i participated at t.

Maintain an exponentially weighted moving average loss:

$$L_{i,t} = (1 - \rho a_{i,t})L_{i,t-1} + \rho a_{i,t}\, \ell_{i,t}.$$

To stop "freezing" skill forever when someone disappears, add a mild staleness pull towards a baseline L_0:

$$L_{i,t} \leftarrow (1 - \kappa)L_{i,t} + \kappa L_0 \quad \text{on rounds with } a_{i,t} = 0.$$

This is the simplest way to reflect the intermittent-contributions setting: absence means "no new evidence", but you still gradually revert confidence.

Map loss state to a bounded skill σ

A monotone bounded map is:

$$\sigma_{i,t} = \sigma_{\min} + (1 - \sigma_{\min}) \exp(-\gamma L_{i,t}), \qquad \sigma_{i,t} \in [\sigma_{\min}, 1].$$

Good behaviour:

improving accuracy $\Downarrow$ $L_{i,t}$ increases $\sigma_{i,t}$,

very poor accuracy $\Uparrow$ $L_{i,t}$ pushes $\sigma_{i,t}$ towards $\sigma_{\min}$,

$\sigma_{\min} > 0$ ensures some minimum exposure via $m_{i,t} \geq \lambda b_{i,t}$.

Vitali-Pinson's online learning is designed to produce relative combination weights for an aggregate forecast under missingness, not an absolute per-user reputation:

They define an availability vector $\alpha_t \in \{0,1\}^N$ and aggregate as

$$\hat{x}^t(\alpha_t) = (w_t + D_t\alpha_t)^\top x^t(\alpha_t),$$

so the effective weights change mechanically with who is present. That makes any "weight" inherently context-dependent, not a stable user skill.

They update w_t via online gradient descent with projection onto a feasible set W (typically a simplex-type constraint):

$$w_{t+1} = \Pi_W(w_t - \eta_t g_t)\,,$$

so weights are normalised and competing. If one person's weight rises, others fall, even if everyone improved in an absolute sense. - Pre-round reputation $\sigma_{i,t}$ decides how much exposure you are allowed to deploy this round.

- Within-round scoring decides who wins this round.

If those two jobs use different information, you are not double counting.

It is not double counting because $\sigma_{i,t}$ is fixed before round t and is computed only from past realised forecast losses (for example CRPS), whereas the settlement payoff in round t uses only current-round objects $(r_{i,t}, m_{i,t}, \omega_t)$. It is compounding because good past accuracy increases future exposure m, so when you are accurate again you win more (and if you are wrong you lose more) since the same relative-performance term is multiplied by a larger m.

Objects:

- Outcome $y_t \in [0,1]$
- Each forecaster i submits a report $r_{i,t} \in [0,1]$ (in MVP: a point forecast)
- Participation indicator $\alpha_{i,t} \in \{0,1\}$ (0 = present, 1 = missing)
- Cash deposit $b_{i,t} \geq 0$
- Skill $\sigma_{i,t} \in [\sigma_{\min}, 1]$ computed before reports in round t

Effective wager

$$m_{i,t} = b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big)$$

and if missing, you force $m_{i,t} = 0$.

Refund:

$$\text{refund}_{i,t} = b_{i,t} - m_{i,t}$$

Loss (MAE):

$$\ell_{i,t} = |r_{i,t} - y_t|$$

Score (bounded):

$$s_{i,t} = \max\bigl(0, \min\bigl(1, 1 - \ell_{i,t}/c\bigr)\bigr)$$

Competitive payout (Raja-style weighted score):

$$\Pi_{i,t} = m_{i,t}\bigl(1 + s_{i,t} - \bar{s}_t\bigr), \qquad \bar{s}_t = \frac{\sum_j m_{j,t} s_{j,t}}{\sum_j m_{j,t}}$$

Profit:

$$\pi_{i,t} = \Pi_{i,t} - m_{i,t} = m_{i,t}\bigl(s_{i,t} - \bar{s}_t\bigr)$$

EWMA loss tracker:

$$L_{i,t+1} = \begin{cases} (1-\rho)L_{i,t} + \rho\,\ell_{i,t} & \alpha_{i,t} = 0 \\ L_{i,t} & \alpha_{i,t} = 1 \end{cases}$$

and skill map:

$$\sigma_{i,t} = \sigma_{\min} + (1 - \sigma_{\min})\exp(-\gamma L_{i,t})$$

Group A: Domain and shape checks (prevent silent nonsense)

(A1) Shape mismatch Ensures every per-player array is length n. Without this, numpy broadcasting can produce a result that "looks fine" but is wrong.

(A2) $y_t \in [0, 1]$ and active reports in $[0, 1]$ You are relying on:

MAE $\in [0, 1]$ when $y, r \in [0, 1]$

score $s = 1 - \ell/c$ staying in $[0, 1]$ with $c = 1$

So this check is what makes bounded-liability arguments valid for the MVP.

(A3) $c > 0$ Avoids division by zero and meaningless scores.

(A4) Finite values Stops NaNs from contaminating skill and payout updates.

Group B: Skill and exposure checks (validate "stake + skill" design exactly)

(B1) $\sigma \in [\sigma_{\min}, 1]$ This validates the intended skill range. It is required to prove the exposure bounds:

$$\lambda b \le m \le b \text{ (active)}.$$

(B2) deposits non-negative and missing deposits are 0 This enforces chosen convention: missing player is not in the round financially.

(B3) Core identity: effective wager formula You explicitly check

$$m_{i,t} = b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big)$$

(and mask missing). This is not just sanity: it is the difference between testing "your system" vs "some other system". nd

You check:

$$\sum_i \Pi_{i,t} = \sum_i m_{i,t}$$

with careful handling when $\sum m \approx 0$. This is the single most important settlement-layer check. If this fails, the mechanism is not self-financed.

(C5) Zero-sum profits

You compute $\pi = \Pi - m$ and check:

$$\sum_i \pi_{i,t} = 0.$$

This is implied by (C4), but it catches different implementation mistakes (for example if profit() is wrong or if you have a mismatch between what you call "wager" and what you pass to profit).

(C6) Single participant with positive wager

If only one player has $m > 0$, then weighted average score equals their score, so:

$$\pi_i = m_i(s_i - \bar{s}) = 0, \qquad \Pi_i = m_i.$$

This catches errors in the computation of $\bar{s}$ and the denominator.

(C7) Profit bounds

You check:

$$-m_i \le \pi_i \le m_i$$

for active $m_i > 0$. This holds because $s, \bar{s} \in [0, 1]$. This is your MVP version of "bounded loss".

(C8) Total cashflow check including refunds

You check:

$$\sum_i b_{i,t} = \sum_i \mathrm{refund}_{i,t} + \sum_i \Pi_{i,t}.$$

This validates the full cash pipeline: deposits come in, part refunded immediately, the rest redistributed through the pool.

Group D: Online learning checks (prove intermittency is handled correctly)

(D1) EWMA update formula for active Checks:

$$L_{i,t+1} = (1-\rho)L_{i,t} + \rho\,\ell_{i,t}.$$

So you know the learning layer matches the intended equation.

(D2) No update for missing Checks:

$$L_{i,t+1} = L_{i,t} \quad \text{if } \alpha_{i,t} = 1.$$

This is exactly the Vitali-style "only update when present" rule at MVP level.

1) Within-round: if you do not show up, you do not matter that round

Participation indicator: $a_{i,t} \in \{0,1\}$.

Active set: $I_t = \{i : a_{i,t} = 1\}$.

If $a_{i,t} = 0$: set $b_{i,t} = m_{i,t} = \hat{\Pi}_{i,t} = 0$. So no forecast, no influence, no payout.

2) Across-round: your skill becomes stale when you are absent

Maintain a per-seller performance state $L_{i,t}$ (EWMA of loss).

If present ($a_{i,t} = 1$):

$$L_{i,t} = (1-\rho)L_{i,t-1} + \rho\,\ell_{i,t}.$$

If absent ($a_{i,t} = 0$) "staleness decay to a prior":

$$L_{i,t} = (1-\kappa)L_{i,t-1} + \kappa L_0.$$

Meaning: with no new evidence, your state drifts back towards a baseline L_0 at rate κ, so you cannot keep a permanently high reputation by disappearing.

Map to bounded skill:

$$\sigma_{i,t+1} = \sigma_{\min} + (1-\sigma_{\min})e^{-\gamma L_{i,t}}.$$

OnlineV2 implementation overview

Purpose and core objects

OnlineV2 simulates an online prediction market over rounds $t = 1, \ldots, T$ with:

- forecasters $i \in \{1, \ldots, n\}$ who submit reports (point or quantile-based),
- realised outcome $y_t \in [0,1]$ each round,

- a pre-round skill signal $\sigma_{i,t} \in [\sigma_{\min}, 1]$ computed from past loss only,
- an effective wager $m_{i,t} = b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big)$ from deposit $b_{i,t}$,
- a self-financed weighted-score settlement (optionally with an extra utility pool),
- an aggregate forecast $\hat{r}_t$ formed as a weighted blend of reports using normalised effective wagers.

Package layout

```
onlinev2/
  mvp.py                    Core simulation loop + invariant checks + tests
  experiments.py            Experiment orchestration, CSV output, plotting hoo
  data_generation/
    data_generation.py      Truth y_t, forecaster reports, client reports
    inputs.py               Missingness alpha_{i,t}, deposits b_{i,t}
  online_algorithms/
    online_skill.py         EWMA loss tracker and loss -> sigma map
    effective_wager.py      Effective wager and refund helpers
    ogd_simplex.py          Optional (not used in the main loop)
  payoff/
    scoring.py              MAE, pinball, CRPS-hat, bounded score mapping
    payoff.py               Settlement, utility term, aggregation
  scripts_r/                R plotting scripts for CSV outputs
  outputs/                  Experiment artefacts (CSVs, figures)
```

Per-round data flow

Core modules

Data generation (`data_generation/data_generation.py`). Generates synthetic y_t and forecaster reports:

- point reports: $r_{i,t} = \mathrm{clip}(y_t + \varepsilon_{i,t}, 0, 1)$ with forecaster-specific noise,
- quantile reports: $q_{i,t}(\tau)$ built from a location-scale form and enforced monotone in τ,
- client report (optional) used to define utility eligibility when $U_t > 0$.

Inputs (`data_generation/inputs.py`).

- missingness $\alpha_{i,t} \in \{0, 1\}$ sampled i.i.d. with $P(\alpha_{i,t} = 1) = p_{\text{miss}}$ and forced to have at least one participant per round,
- deposits $b_{i,t} \geq 0$ (typically exponential), set to 0 when $\alpha_{i,t} = 1$.

Skill updates (`online_algorithms/online_skill.py`). Tracks an EWMA loss state $L_{i,t}$ and maps it to skill:

$$L_{i,t} = \begin{cases} (1-\rho)L_{i,t-1} + \rho\,\ell_{i,t} & \text{if present} \\ (1-\kappa)L_{i,t-1} + \kappa L_0 & \text{if missing} \end{cases}$$

$$\sigma_{i,t} = \sigma_{\min} + (1-\sigma_{\min})\exp(-\gamma L_{i,t-1}) \in [\sigma_{\min}, 1].$$

This enforces pre-round timing: $\sigma_{i,t}$ depends only on information up to $t-1$.

Effective wager (`online_algorithms/effective_wager.py`).

$$m_{i,t} = b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big), \qquad \text{refund}_{i,t} = b_{i,t} - m_{i,t}.$$

Scoring (`payoff/scoring.py`). Two evaluation modes:

- point: $\ell_{i,t} = |r_{i,t} - y_t|$, with bounded score $s_{i,t} = \text{clip}(1 - \ell_{i,t}/c, 0, 1)$ (often $c = 1$),
- quantiles: CRPS approximation via pinball loss, $\ell_{i,t} = \widehat{\text{CRPS}}(q_{i,t}, y_t)$, then

$$s_{i,t} = \text{clip}\left(1 - \frac{\ell_{i,t}}{c},\ 0,\ 1\right)$$

  with a mode-specific c (often $c = 2$ for CRPS-hat bounds).

Settlement and aggregation (`payoff/payoff.py`). Let $\bar{s}_t = \frac{\sum_j m_{j,t} s_{j,t}}{\sum_j m_{j,t}}$ over active m.

- skill payoff (self-financed weighted-score):

$$\Pi^{\text{skill}}_{i,t} = m_{i,t}\big(1 + s_{i,t} - \bar{s}_t\big), \qquad \sum_i \Pi^{\text{skill}}_{i,t} = \sum_i m_{i,t},$$

- optional utility payoff: allocate U_t only to those beating the client score $s_{c,t}$ using weights proportional to $\tilde{s}_{i,t} m_{i,t}$, where $\tilde{s}_{i,t} = s_{i,t}\mathbf{1}\{s_{i,t} > s_{c,t}\}$,
- cashout and profit:

$$\text{cashout}_{i,t} = \text{refund}_{i,t} + \Pi^{\text{skill}}_{i,t} + \Pi^{\text{util}}_{i,t}, \qquad \pi_{i,t} = \big(\Pi^{\text{skill}}_{i,t} + \Pi^{\text{util}}_{i,t}\big) - m_{i,t},$$

- aggregation: normalise $\hat{m}_{i,t} = m_{i,t}/\sum_j m_{j,t}$ and combine reports linearly (point or quantile-wise). If $\sum_j m_{j,t} \approx 0$, return a fallback (client report if available) or 0.

Core loop: `mvp.run_simulation`

The simulation initialises (y_t), reports, $(\alpha_{i,t})$, and $(b_{i,t})$, then for each round:

1. compute $\sigma_{i,t}$ from $L_{i,t-1}$,
2. compute losses and scores (missing agents set to 0),
3. settle the round (compute $m_{i,t}$, refunds, payoffs, profit),
4. aggregate forecasts using $\hat{m}_{i,t}$,
5. update $L_{i,t}$ using the EWMA rule above,
6. optionally store full histories (skill, wagers, losses, scores, payouts).

Invariant checks and tests (in `mvp.py`)

Per-round and run-level checks enforce:

- domain checks: $y_t \in [0,1]$ and (active) reports in $[0,1]$,
- $\sigma_{i,t} \in [\sigma_{\min}, 1]$ and the exact effective wager identity,
- missing agents excluded: $b_{i,t} = m_{i,t} = \Pi_{i,t} = 0$ when $\alpha_{i,t} = 1$,
- budget balance:

$$\sum_i \Pi_{i,t}^{\text{skill}} = \sum_i m_{i,t}, \qquad \sum_i \left(\Pi_{i,t}^{\text{skill}} + \Pi_{i,t}^{\text{util}}\right) = \sum_i m_{i,t} + U_t,$$

- full cashflow identity:

$$\sum_i b_{i,t} = \sum_i \text{refund}_{i,t} + \sum_i \Pi_{i,t}^{\text{skill}}, \qquad \sum_i \text{cashout}_{i,t} = \sum_i b_{i,t} + U_t,$$

- profit bounds for active agents: $-m_{i,t} \le \pi_{i,t} \le m_{i,t}$ (since $s_{i,t}, \bar{s}_t \in [0,1]$),
- EWMA update matches the stated present and missing cases.

Experiments layer (`experiments.py`)

Experiments vary parameters, run simulations, and write CSVs for plotting, including:

- settlement sanity checks (budget balance, non-negativity, symmetry),
- skill and wager intermittency diagnostics (time series of σ, m, m/b, profit),

- forecast aggregation comparisons (stake-only vs skill-only vs blended vs equal-weight),
- calibration diagnostics for quantiles (empirical coverage),
- sybil-style stake splitting stress tests (single identity vs split identities),
- parameter sweeps over $(\lambda, \sigma_{\min})$ and summary metrics.

Conceptual dependency graph

```
experiments.py
  -> mvp.run_simulation, mvp.run_all_tests
  -> payoff.scoring, payoff.payoff

mvp.run_simulation
  -> data_generation (truth, reports, client report)
  -> inputs (missingness, deposits)
  -> online_skill (EWMA, loss->sigma)
  -> scoring (loss, score)
  -> payoff (settlement, aggregation)
```

Experiments and output interpretation

Common core (most experiments)

Key objects (round t).

- Deposit: $b_{i,t} \geq 0$.
- Skill: $\sigma_{i,t} \in [\sigma_{\min}, 1]$, computed *before* round t from past loss state.
- Effective wager (with missingness masking):
$$m_{i,t} = b_{i,t}\big(\lambda + (1-\lambda)\sigma_{i,t}\big), \qquad m_{i,t} = 0 \text{ if } \alpha_{i,t} = 1.$$
- Refund:
$$\text{refund}_{i,t} = b_{i,t} - m_{i,t}.$$
- Score from loss:
$$s_{i,t} = \text{clip}\left(1 - \frac{\ell_{i,t}}{c},\, 0,\, 1\right),$$
with $c = 1$ (MAE) or $c = 2$ (CRPS-hat).

Settlement (self-financed weighted-score). Let
$$\bar{s}_t = \frac{\sum_j m_{j,t} s_{j,t}}{\sum_j m_{j,t}}, \qquad \Pi^{\text{skill}}_{i,t} = m_{i,t}\left(1 + s_{i,t} - \bar{s}_t\right).$$

Profit (net of effective wager):

$$\pi_{i,t} = \Pi_{i,t}^{\text{skill}} - m_{i,t}.$$

Budget balance per round:

$$\sum_i \Pi_{i,t}^{\text{skill}} = \sum_i m_{i,t}.$$

1) Settlement sanity

What it does. Validates that the settlement rule is budget-balanced, payouts are non-negative, and the equal-score tie case yields zero profit.

Mechanics. Samples synthetic rounds with random m, random s, applies missingness masking, computes

$$\Pi_i = m_i(1 + s_i - \bar{s}), \qquad \pi_i = \Pi_i - m_i,$$

then checks budget balance and non-negativity. Also checks the equal-score tie case.

Outputs. `settlement_sanity.csv`, `settlement_sanity_series.csv`, `settlement_sanity_h` `settlement_sanity.png`.

Metric	Value	Interpretation
n_{rounds}	500	500 independent synthetic rounds.
mean_profit	$\approx 3.53 \times 10^{-17}$	Effectively 0, consistent with zero-sum profit under self-financing.
std_profit	≈ 1.42	Profit variance is large even though the mean is 0.
mean_roi	≈ -0.0086	Slightly negative average ROI, consistent with averaging a ratio under heterogeneous stakes.
mean_abs_budget_gap	$\approx 1.94 \times 10^{-15}$	Floating-point noise scale.
max_abs_budget_gap	$\approx 2.84 \times 10^{-14}$	Worst-case still at numerical tolerance scale.
equal_scores_ok	1	Tie case passes.
equal_scores_max_abs_profit	0.0	Exactly zero profit under equal scores (within computation).

Your summary metrics and interpretation. **Conclusion:** settlement is correct to numerical tolerance.

2) Skill–wager under intermittency

What it does. Shows how $\sigma_{i,t}$ and $m_{i,t}$ evolve when agents intermittently do not participate.

Mechanics. Runs a simulation with missingness, exports time series for selected agents: $\sigma_{i,t}$, $m_{i,t}$, $b_{i,t}$, $\alpha_{i,t}$, the ratio $m_{i,t}/b_{i,t}$ on active rounds, and profit trajectories. Also records the theoretical lower bound ratio

$$\min_ratio = \lambda + (1 - \lambda)\sigma_{\min}.$$

Outputs. `skill_wager_intermittency.csv`, `skill_wager_intermittency_plot.csv`, `skill_wager_intermittency_params.csv`, `skill_wager_intermittency_agent_0.png` (and 1, 2).

Agent	mean_sigma	mean_wager	total_profit
0	0.603	0.568	-0.975
1	0.684	0.535	-0.388
2	0.759	0.561	$+0.184$
3	0.788	0.639	$+0.544$
4	0.829	0.586	$+0.722$
5	0.707	0.553	-0.088

Your per-agent summary and interpretation. With $\lambda = 0.3$ and $\sigma_{\min} = 0.1$,

$$\min_ratio = 0.3 + 0.7 \cdot 0.1 = 0.37,$$

so on active rounds the ratio satisfies $m_{i,t}/b_{i,t} \in [0.37, 1]$.

Conclusion: intermittency masking and the m/b bounds behave as intended; higher mean skill aligns with higher profit in this realisation.

3) Forecast aggregation

What it does. Compares aggregate forecast quality (CRPS-hat) under four weighting schemes:

- blended weights (m),
- equal weights,
- stake-only weights (b),
- skill-only weights ($b\sigma$).

Mechanics. On each valid round, compute each aggregate $\hat{q}_t(\tau)$, evaluate CRPS-hat, and track rolling and cumulative means.

Outputs. `forecast_aggregation.csv`, `forecast_aggregation.png`.

Your end-of-run ordering (cumulative CRPS-hat).

- equal: ≈ 0.02253 (best),
- skill-only: ≈ 0.02333,
- blended: ≈ 0.02341,
- stake-only: ≈ 0.02357 (worst).

Conclusion: in this run, equal weighting is most accurate; mechanism weighting underperforms equal weighting on CRPS-hat (a performance result, not a correctness issue).

4) Calibration diagnostics (quantile reliability)

What it does. Checks quantile calibration of the aggregate distribution by estimating

$$\hat{p}(\tau) = \Pr\big(y_t \le \hat{q}_t(\tau)\big),$$

with perfect calibration given by $\hat{p}(\tau) = \tau$.

Outputs. `calibration_reliability.csv`, `calibration_reliability.png`.

τ	$\hat{p}(\tau)$	n_{valid}
0.1	0.0033	300
0.25	0.0733	300
0.5	0.4633	300
0.75	0.9267	300
0.9	0.9867	300

Your reliability table and interpretation. Low quantiles have $\hat{p}(\tau) \ll \tau$ and high quantiles have $\hat{p}(\tau) \gg \tau$, indicating systematic tail miscalibration in this run.

Conclusion: calibration, especially in the tails, is the main modelling weakness in these outputs.

5) Sybil / identity split

What it does. Tests whether splitting one identity into k identities (same total stake and score) increases total profit.

Mechanics. Compares focal profit under single vs split:

$$\Delta = \pi^{\text{split}} - \pi^{\text{single}}, \qquad \text{ratio} = \frac{\pi^{\text{split}}}{\pi^{\text{single}}},$$

then aggregates deltas and confidence intervals across trials.

Outputs. `sybil_profit_ratio.csv`, `sybil_profit_ratio.png`.

Your result and interpretation. Ratios are essentially 1 across k, with Δ around 10^{-16} and confidence intervals containing 0.

Conclusion: identity splitting provides no advantage (residuals are floating-point noise).

6) Parameter sweep $(\lambda, \sigma_{\min})$

What it does. Explores the trade-off between forecast accuracy (mean CRPS-hat), payout inequality (Gini of shifted total profits), and market activity (fraction of meaningful rounds).

Mechanics. For each grid point:

- mean CRPS-hat over valid rounds,
- Gini of $x_i = \pi_i - \min_j \pi_j$,
- fraction of rounds with $\sum_i m_{i,t} > \varepsilon$.

Outputs. `parameter_sweep.csv`, `parameter_sweep_crps.png`, `parameter_sweep_gini.png`, `parameter_sweep_frac_meaningful.png`.

Your summary and interpretation.

- mean CRPS-hat ranges roughly 0.02686 to 0.02717, best near low λ and low $\sigma_{\min}$.
- Gini ranges roughly 0.2200 to 0.2298, highest at low λ and low $\sigma_{\min}$.
- fraction meaningful is 1.0 throughout your sweep.

Conclusion: lower λ and lower $\sigma_{\min}$ marginally improve accuracy but increase inequality.

7) Fixed deposit: isolate the skill effect

What it does. Removes deposit variation so m varies only through σ.

Mechanics. Set $b_{i,t} = 1$ each round, so

$$\frac{m_{i,t}}{b_{i,t}} = \lambda + (1 - \lambda)\sigma_{i,t}.$$

Plot $\sigma(t)$ and m/b for selected agents.

Outputs. `fixed_deposit_skill_effect_plot.csv`, `fixed_deposit_skill_effect_params.c` `fixed_deposit_skill_effect.png`.

Your summary and interpretation. With $\lambda = 0.3$ and $\sigma_{\min} = 0.1$, the minimum ratio is 0.37, and m/b mirrors σ via $0.3 + 0.7\sigma$.

Conclusion: the skill lever is applied cleanly when deposit noise is removed.

8) Scoring validation

What it does. Runs the test battery under both scoring modes (MAE and CRPS-hat) and records invariants.

Output. `scoring_validation.csv`.

Your summary and interpretation. Booleans are `True` and budget gaps and profit sums remain around 10^{-15}.

Conclusion: both scoring paths are numerically stable, bounded, and consistent with settlement invariants.

Cross-experiment conclusions

1. Correctness and invariants are solid (budget balance at $\sim 10^{-14}$, equal-score tie behaves as expected).
2. Sybil splitting shows no advantage (ratios ≈ 1, deltas $\sim 10^{-16}$).
3. Aggregation accuracy is not beating equal weights in this run (equal-weight has the lowest cumulative CRPS-hat).
4. Calibration is the main modelling issue here, with strong tail miscalibration in the reliability table.
5. The parameter sweep shows an accuracy versus equality trade-off as $(\lambda, \sigma_{\min})$ vary.

Actionable next step. Keep settlement and sybil structure unchanged, focus on quantile generation and calibration (either improve forecaster quantiles or add a calibration layer for aggregated quantiles).

Tests in `onlinev2` and results

Where tests live and how they run

- **Scoring unit tests (4)** live in `mvp.py`. They are run from `experiments.py`. Results are written to `outputs/tests/test_results.txt` as PASS/FAIL.
- **Full test suite** is run by `run_all_tests()` in `mvp.py` (the same 4 scoring tests plus additional unit tests and snapshot-based tests). Results are written to `outputs/experiments/scoring_va`

as booleans (True/False) plus numeric metrics.

Results summary

Scoring unit tests (`test_results.txt`).

```
Scoring unit tests
================
pinball_nonneg: PASS
crps_nonneg: PASS
crps_perfect_better: PASS
crps_bound: PASS
```

Full suite (`scoring_validation.csv`). All boolean tests below were **True** for both `point_mae` and `quantiles_crps` in the recorded run. Numeric metrics are reported in Table 2.

Table 2: Numeric metrics from `scoring_validation.csv`.

Metric	`point_mae`	`quantiles_crps`
max_abs_budget_gap	1.78×10^{-15}	3.55×10^{-15}
sum_total_profit	-2.78×10^{-15}	6.33×10^{-15}
corr_noise_mean_sigma	-0.952	-0.979
corr_noise_total_profit	-0.936	-0.960

The four scoring unit tests (written to `test_results.txt`)

These validate the loss and score functions in `payoff/scoring.py`.

`pinball_nonneg`
Property. Pinball loss is non-negative for $y, q \in [0, 1]$ and $\tau \in (0, 1)$.

$$L^\tau(y, q) = \begin{cases} \tau(y - q) & \text{if } y \geq q, \\ (1 - \tau)(q - y) & \text{if } y < q, \end{cases} \qquad L^\tau(y, q) \geq 0.$$

Implementation (in `payoff/scoring.py`).

```
def pinball_loss(y, q, tau):
    err = y - q
    return np.where(err >= 0, tau * err, (tau - 1.0) * err)
```

Test (in `mvp.py`).

```
def unit_pinball_nonneg(seed: int = 90) -> bool:
    rng = np.random.default_rng(seed)
    y = rng.uniform(0.0, 1.0, size=50)
```

```
    q = rng.uniform(0.0, 1.0, size=50)
    for tau in [0.1, 0.5, 0.9]:
        L = pinball_loss(y, q, tau)
        if np.any(L < -1e-12):
            return False
    return True
```

Result. PASS.

`crps_nonneg`

Property. The CRPS approximation (two times the mean pinball loss over quantile levels) is non-negative.

$$\widehat{\mathrm{CRPS}}(y, \{q_{\tau_k}\}_{k=1}^{K}) = \frac{2}{K} \sum_{k=1}^{K} L^{\tau_k}(y, q_{\tau_k}) \geq 0.$$

Test (in `mvp.py`).

```
def unit_crps_nonneg(seed: int = 91) -> bool:
    rng = np.random.default_rng(seed)
    y = float(rng.uniform(0.0, 1.0))
    q = rng.uniform(0.0, 1.0, size=(5, 5))
    taus = np.array([0.1, 0.25, 0.5, 0.75, 0.9])
    L = crps_hat_from_quantiles(y, q, taus)
    return bool(np.all(L >= -1e-12))
```

Result. PASS.

`crps_perfect_better`

Property. A forecast matching the realised outcome at all quantiles should achieve weakly lower CRPS than a uniformly shifted forecast.

Test (in `mvp.py`).

```
def unit_crps_perfect_better(seed: int = 92) -> bool:
    y = 0.5
    taus = np.array([0.1, 0.25, 0.5, 0.75, 0.9])
    q_perfect = np.full((3, 5), 0.5)
    q_shifted = np.full((3, 5), 0.6)
    L_perfect = crps_hat_from_quantiles(y, q_perfect, taus)
    L_shifted = crps_hat_from_quantiles(y, q_shifted, taus)
    return bool(np.all(L_perfect <= L_shifted + 1e-9))
```

Result. PASS.

crps_bound

Property. For $y, q \in [0, 1]$, the implemented $\widehat{\mathrm{CRPS}}$ stays bounded by 2 (within numerical tolerance), supporting the score scaling $s = \mathrm{clip}(1 - \ell/c, 0, 1)$ with $c = 2$.

Test (in mvp.py).

```
def unit_crps_bound(seed: int = 93) -> bool:
    rng = np.random.default_rng(seed)
    for _ in range(20):
        y = float(rng.uniform(0.0, 1.0))
        q = rng.uniform(0.0, 1.0, size=(4, 5))
        taus = np.array([0.1, 0.25, 0.5, 0.75, 0.9])
        L = crps_hat_from_quantiles(y, q, taus)
        if np.any(L > 2.0 + 1e-9):
            return False
    return True
```

Result. PASS.

Full suite (booleans and what each checks)

The full suite is executed by `run_all_tests()` and reported in `scoring_validation.csv`.

Snapshot-based tests (use a real round snapshot)

wager_scaling_test. **Property.** Scaling all deposits by $k > 0$ scales effective wagers by k and profits by k for active agents (homogeneity in stake).

Result. True for both `point_mae` and `quantiles_crps`.

identity_split_local. **Property.** Splitting one agent into two identities with the same total deposit and identical (s, σ) preserves the combined profit of the split identities, and leaves other agents unchanged.

Result. True for both `point_mae` and `quantiles_crps`.

Payoff and settlement unit tests

All were True for both modes in the recorded run:

- `two_player_closed_form` (two-player profit closed form, zero-sum symmetry)
- `equal_score_zero_profit` (constant scores imply zero profit transfers)
- `permutation_invariance` (permuting agents permutes outputs identically)
- `zero_wager_dummy` (adding a zero-wager agent changes nothing)
- `score_shift_invariance` (common shift of scores leaves profits unchanged when still

in $[0, 1]$)

- `roi_bounds` ($\text{profit}_i/m_i \in [-1, 1]$ for active $m_i > 0$)
- `near_zero_total_wager` (finite outputs and budget balance under tiny wagers)
- `missing_excluded` (missing agent has $m = s = \Pi = 0$, budget balance over active set)
- `all_missing_zero_payout` (all missing implies zero payouts)
- `zero_element_all_fields` (zero deposit implies zero settlement fields at that index)
- `budget_identity_with_utility` (with $U > 0$, $\sum_i \Pi_i = \sum_i m_i + \sum_i \Pi_i^{\text{util}}$)

Scoring, bounds, timing, and smoke tests
All were True for both modes in the recorded run:

- `mae_median_sanity` (empirical MAE minimised near the median, 0.5 for uniform)
- `score_bounds` (score mapping stays in $[0, 1]$ under stressed losses)
- `sigma_pre_round_timing` (σ_t depends only on L_{t-1}, not current-round data)
- `quantiles_with_utility_smoke` (short quantiles run with $U > 0$ remains budget balanced)
- `extreme_params_smoke` (extreme parameters avoid numerical failures and remain budget balanced)

Interpretation of numeric metrics

- **Budget balance.** `max_abs_budget_gap` is at machine precision ($\approx 10^{-15}$), consistent with self-financing each round.
- **Zero-sum profits.** `sum_total_profit` is essentially 0, consistent with profit being redistribution of the wager pool (utility aside).
- **Noise correlations.** Strong negative correlations indicate higher noise implies lower mean skill and lower total profit, matching the intended skill signal behaviour.

Summary table

Table 3: Test outcomes for the recorded run.

Test	`point_mae`	`quantiles_crps`
`extreme_params_smoke`	True	True

Data-generating processes (DGPs)

Exogenous-truth family (truth drawn first, independent of reports)
Baseline (`baseline`). Each round draws a scalar truth

$$y_t \sim \mathrm{Unif}(0, 1).$$

Forecaster i then produces a noisy point report by adding zero-mean Gaussian noise and clipping back to $[0, 1]$:

$$r_{i,t} = \mathrm{clip}\big(y_t + \varepsilon_{i,t},\, 0,\, 1\big), \qquad \varepsilon_{i,t} \sim \mathcal{N}(0, \tau_i^2).$$

The per-forecaster noise levels τ_i are drawn once (log-normal in the implementation) to induce heterogeneous forecast quality. Conceptually, this corresponds to an unbiased signal corrupted by additive noise; in the absence of clipping, the classical optimal linear combiner is inverse-variance weighting,

$$w_i^\star \propto \frac{1}{\tau_i^2},$$

with the caveat that clipping introduces boundary effects near 0 and 1.

Latent-fixed (`latent_fixed`). Each round draws a latent Gaussian state

$$Z_t \sim \mathcal{N}(0, \sigma_Z^2), \qquad y_t = \Phi(Z_t),$$

mapping to the unit interval via a probit link. Forecaster i observes a noisy (optionally biased) signal

$$X_{i,t} = Z_t + b_i + \tau_i \varepsilon_{i,t}, \qquad \varepsilon_{i,t} \sim \mathcal{N}(0, 1),$$

and reports the Bayes posterior mean in latent space, passed through the same link:

$$r_{i,t} = \Phi\big(\mu_{i,t}^Z\big)\,, \qquad \mu_{i,t}^Z = \frac{\sigma_Z^2}{\sigma_Z^2 + \tau_i^2}\,(X_{i,t} - b_i).$$

This DGP is Bayes-consistent by construction: more precise signals (smaller τ_i) yield stronger shrinkage-corrected estimates and tighter predictive uncertainty. The posterior variance in latent space is

$$v_i = \frac{\sigma_Z^2 \tau_i^2}{\sigma_Z^2 + \tau_i^2},$$

so a natural precision proxy is $1/v_i$.

Endogenous-truth family (truth depends on forecaster signals)
Shared backbone (all aggregation DGPs). A persistent latent mean μ_t evolves as an AR(1):

$$\mu_t = \rho\, \mu_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, \sigma_{\text{state}}^2), \qquad \rho \approx 0.98.$$

Forecaster i has a latent signal

$$x_{i,t} = \mu_t + \delta_{i,t} + \tau_i \xi_{i,t}, \qquad \xi_{i,t} \sim \mathcal{N}(0, 1),$$

and reports

$$r_{i,t} = \Phi(x_{i,t}).$$

Latent truth is generated from the same forecaster signals via structural weights w:

$$y_t^{\text{latent}} = \sum_i w_i x_{i,t} + \sigma_\varepsilon \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \qquad y_t = \Phi\big(y_t^{\text{latent}}\big)\,.$$

Because y_t is constructed from $\{x_{i,t}\}_i$, there is a well-defined structural weight vector w that a learning algorithm can, in principle, attempt to recover. This contrasts with exogenous-truth DGPs, where "optimal weights" are prediction-optimal but do not correspond to structural generative coefficients.

Aggregation Method 1 (`aggregation_method1`). This corresponds to the shared backbone with no idiosyncratic mean term,

$$\delta_{i,t} = 0.$$

All agents share the same drifting centre μ_t and differ only through their noise scales τ_i (and through the fixed structural weights w_i used to form truth).

Aggregation Method 2 (`aggregation_method2`). This is currently identical to Method 1 and is registered as a separate name (an alias) so that alternative endogenous structures can be substituted later without modifying downstream experiment plumbing.

Aggregation Method 3 (`aggregation_method3`). This matches the shared backbone but introduces per-forecaster mean shocks, $\delta_{i,t} \neq 0$ (modelled as Gaussian shocks or drift around μ_t). Forecasters now disagree for two reasons: observation noise $\tau_i \xi_{i,t}$ and genuine shifts in their latent centres $\delta_{i,t}$. This increases difficulty for both forecasting and weight recovery because the relationship between any single forecaster and the realised truth becomes less stable over time.

Two broad families of DGPs

Two broad families are distinguished by whether the outcome is generated independently of the forecasters.

Exogenous truth. The outcome is generated by an external process first, and forecasters subsequently attempt to predict it.

Endogenous truth. The outcome is partially constructed from the forecasters' latent signals (a "wisdom of the crowd" world), so the realised truth is not independent of the forecasting population.

Family 1: Exogenous truth

1) Baseline (`baseline`). Each round draws a scalar outcome $y_t \in [0,1]$ exogenously (e.g. $y_t \sim \mathrm{Unif}(0,1)$). Forecaster i reports a noisy point forecast of the form

$$r_{i,t} \approx y_t + \text{noise}.$$

In this DGP, forecasters differ only in noise magnitude: a "better" forecaster has smaller noise variance, implying forecasts are closer to y_t on average. This is the simplest forecasting environment, in which all agents aim to predict a fixed external truth.

2) Latent-fixed (`latent_fixed`). Each round draws a latent real-valued state $Z_t \in R$ and maps it into the unit interval via a probit link,

$$y_t = \Phi(Z_t).$$

Forecasters do not observe Z_t directly. Instead, forecaster i receives a noisy signal $X_{i,t}$ about Z_t and reports the statistically optimal estimate of the truth given that signal (equivalently, the Bayes posterior summary under the specified signal model). A "better" forecaster has a less noisy signal, which yields more accurate point estimates and predictive uncertainty that is better calibrated to realised outcomes.

Family 2: Endogenous truth (aggregation DGPs)
All aggregation DGPs share the following structure:

- a latent "state of the world" μ_t evolves slowly over time;
- each forecaster observes a noisy private signal about μ_t;
- the realised truth is generated by taking a weighted average of the forecasters' latent signals, plus additional noise;
- truth and forecasts are mapped into $[0, 1]$ (e.g. via a probit link).

In this family, the realised outcome is not independent: it is explicitly constructed from the forecasters' information.

3) Aggregation Method 1 (`aggregation_method1`). All forecasters are centred on the same underlying state μ_t and differ only in their noise levels. The latent truth is formed as a weighted sum of their signals with fixed structural weights w.

4) Aggregation Method 2 (`aggregation_method2`). This is currently identical to Aggregation Method 1 and is retained as a placeholder name, allowing the endogenous structure to be modified later without changing downstream code.

5) Aggregation Method 3 (`aggregation_method3`). This extends Aggregation Method 1 by adding a forecaster-specific drifting offset to each agent's latent signal. Consequently, forecasters disagree not only due to observation noise but also due to heterogeneous drift in their latent centres. This makes both forecasting and weight learning more challenging because the relationship between an individual forecaster and the realised truth becomes less stable over time.

Exogenous versus endogenous truth: advantages and limitations

Exogenous truth (`baseline`, `latent_fixed`)
Pros.

- **Clean skill definition:** "good" forecasters are those whose reports are consistently closer (in expected loss) to an external outcome y_t.
- **No circularity:** forecasters cannot appear accurate merely because the outcome construction depends on their signals.
- **Best for validation:** supports mechanism-level validation (skill learning, calibration, incentive properties) against an independent ground truth.
- **Interpretability:** enables clearer comparisons of aggregation schemes (e.g. equal-weight, skill-weight, stake-weight) without structural confounds.

Cons.

- **Reduced realism for aggregative outcomes:** less representative when the empirical "truth" is itself an aggregation (e.g. surveys, indices, committee outcomes).
- **No structural weights to recover:** there is no generative weight vector w; any learned weights are prediction-optimal rather than structural or causal.
- **Stake informativeness is imposed:** if stake is treated as informative, this is an additional behavioural assumption, not a property implied by the DGP.

Endogenous truth (aggregation methods)
Pros.

- **Alignment with "wisdom of crowds":** appropriate when realised outcomes behave like a weighted mixture of sources.
- **Structural recovery target:** embeds a ground-truth weight vector w in the DGP, providing a clear objective for weight-recovery experiments.
- **Feedback-loop analysis:** enables study of interactions between influence, learning dynamics, and truth formation over time.

Cons.

- **Circularity and confounding:** a forecaster may appear accurate because the outcome uses their signal (especially when w_i is large), rather than because they are intrinsically precise.
- **Ambiguous skill interpretation:** accuracy and influence are entangled, complicating the interpretation of loss-based "skill" estimates.

- **Model-class dependence:** if truth is linear in latent space and the learner is also linear, recovery may be artificially easy relative to real-world settings.
- **More demanding diagnostics:** temporal dependence and drift increase variance in learning curves, typically requiring longer runs and more seeds for stable conclusions.